Applied Numerical Methods for Civil Engineering

CGN 3405 - 0002

Week 12: Optimization techniques: Part I

Xinyu Chen

Assistant Professor

University of Central Florida

Linear Programming

- **Concept**: Linear programming or linear optimization
- A special case of mathematical programming
- **Definition**: Linear programming is a technique for the optimization of a **linear objective function**, subject to (s.t.) **linear equality and linear inequality constraints**.
- **Description**: Find the **best outcome** (such as maximum profit or lowest cost) in a model where requirements are represented by **linear** relationships.

Mathematical Programming

- **A general formula**:

$$\begin{array}{ll} \min\limits_{x} & f(x) \\ \text{s.t.} & x \in \mathcal{C} \end{array} \quad \text{or} \quad \begin{array}{ll} \max\limits_{x} & f(x) \\ \text{s.t.} & x \in \mathcal{C} \end{array}$$

- **Main components**:
  - Decision variable x
  - Objective function $f(x)$
  - Linear constraint $x \in \mathcal{C}$
  - Feasible set $\mathcal{C}$, e.g., $\mathcal{C} = \{x \mid x \geq 0\}$
- **Format**: Minimize ($\min$) vs. Maximize ($\max$)

Linear Programming

- **Example**:

$$\begin{aligned} \max_{x_1,x_2} \quad & c_1x_1 + c_2x_2 && \text{(linear objective)} \\ \text{s.t.} \quad & a_{11}x_1 + a_{12}x_2 \le b_1 && \text{(linear inequality)} \\ & a_{21}x_1 + a_{22}x_2 \le b_2 && \text{(linear inequality)} \\ & x_1, x_2 \ge 0 && \text{(non-negativity)} \end{aligned}$$

or

$$\begin{aligned} \max_{\boldsymbol{x}} \quad & \boldsymbol{c}^\top \boldsymbol{x} && \text{(linear objective)} \\ \text{s.t.} \quad & \boldsymbol{A}\boldsymbol{x} \le \boldsymbol{b} && \text{(linear inequality)} \\ & \boldsymbol{x} \ge 0 && \text{(non-negativity)} \end{aligned}$$

for any matrix $\boldsymbol{A}$ and vectors $\boldsymbol{b}, \boldsymbol{c}$.

Geometry of Linear Programming

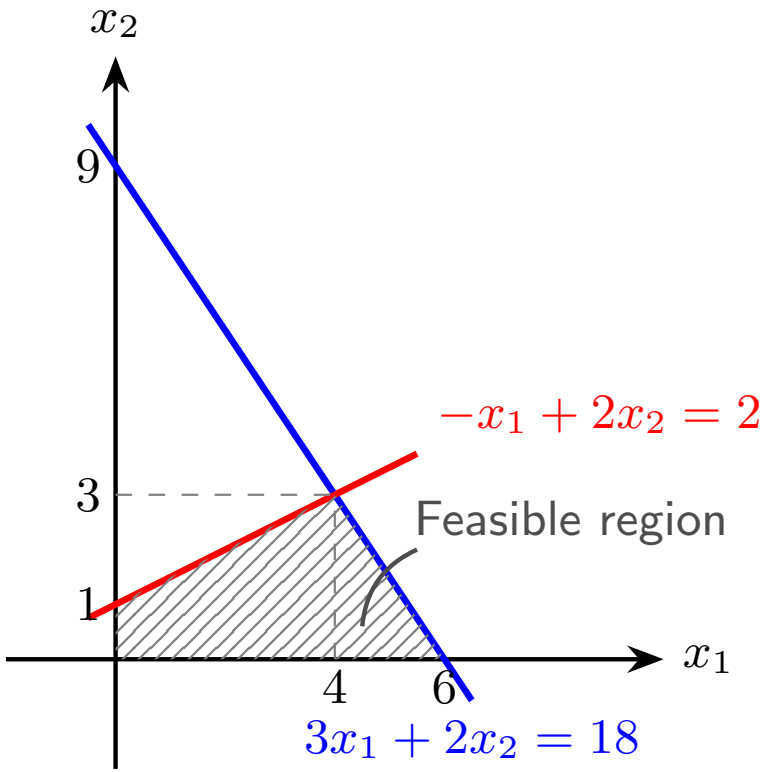
Linear programming:

$$\begin{aligned} \max_{x_1, x_2} \quad & x_1 + x_2 \\ \text{s.t.} \quad & 3x_1 + 2x_2 \leq 18 \\ & -x_1 + 2x_2 \leq 2 \\ & x_1, x_2 \geq 0 \end{aligned}$$

Geometry of Linear Programming

Linear programming:

$$\begin{aligned} \max_{x_1, x_2} \quad & x_1 + x_2 \\ \text{s.t.} \quad & 3x_1 + 2x_2 \leq 18 \\ & -x_1 + 2x_2 \leq 2 \\ & x_1, x_2 \geq 0 \end{aligned}$$



Linear constraints define the **feasible region**, which is a convex polytope.

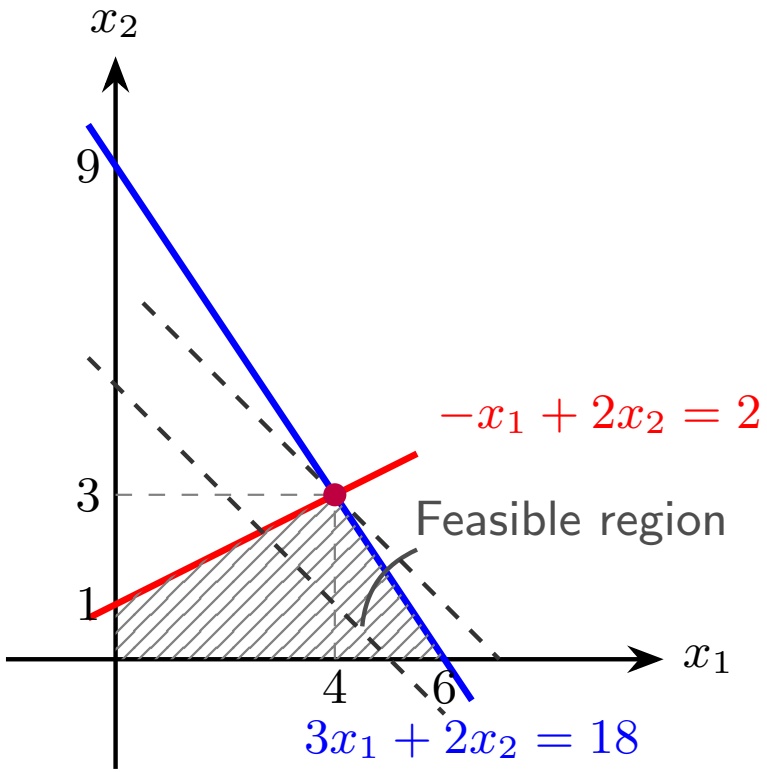
Geometry of Linear Programming

Linear programming:

$$\begin{aligned} \max_{x_1,x_2} \quad & x_1 + x_2 \\ \text{s.t.} \quad & 3x_1 + 2x_2 \leq 18 \\ & -x_1 + 2x_2 \leq 2 \\ & x_1, x_2 \geq 0 \end{aligned}$$

Optimal solution:

$$x_1^\star = 4 \quad x_2^\star = 3$$



Linear constraints define the **feasible region**, which is a convex polytope.

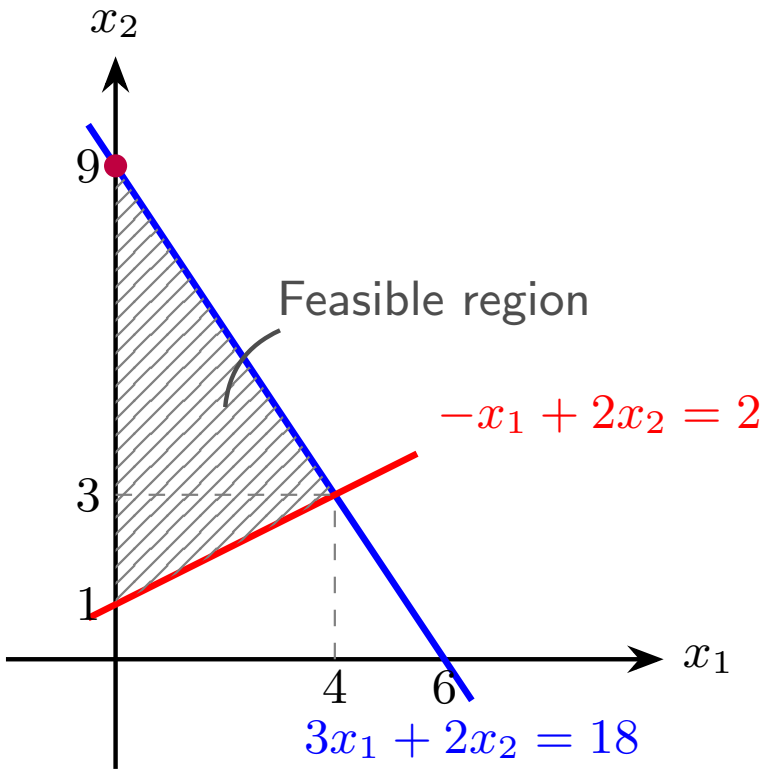
Geometry of Linear Programming

Linear programming:

$$\begin{aligned} \max_{x_1,x_2} \quad & x_1 + x_2 \\ \text{s.t.} \quad & 3x_1 + 2x_2 \leq 18 \\ & -x_1 + 2x_2 \geq 2 \\ & x_1, x_2 \geq 0 \end{aligned}$$

Optimal solution:

$$x_1^\star = 0 \quad x_2^\star = 9$$



Linear constraints define the **feasible region**, which is a convex polytope.

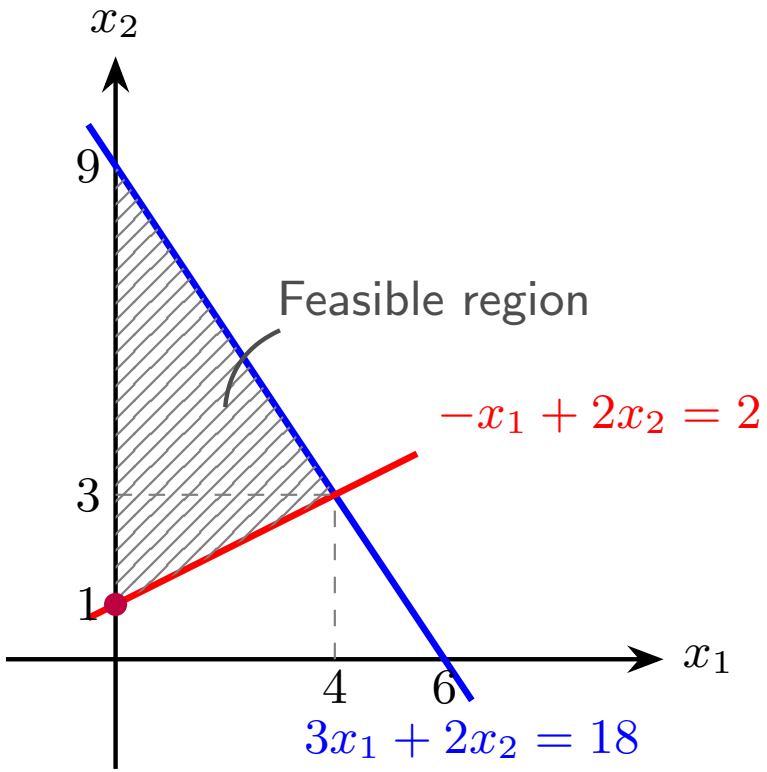
Geometry of Linear Programming

Linear programming:

$$\begin{aligned} \min_{x_1,x_2} \quad & x_1 + x_2 \\ \text{s.t.} \quad & 3x_1 + 2x_2 \leq 18 \\ & -x_1 + 2x_2 \geq 2 \\ & x_1, x_2 \geq 0 \end{aligned}$$

Optimal solution:

$$x_1^\star = 0 \quad x_2^\star = 1$$



Linear constraints define the **feasible region**, which is a convex polytope.

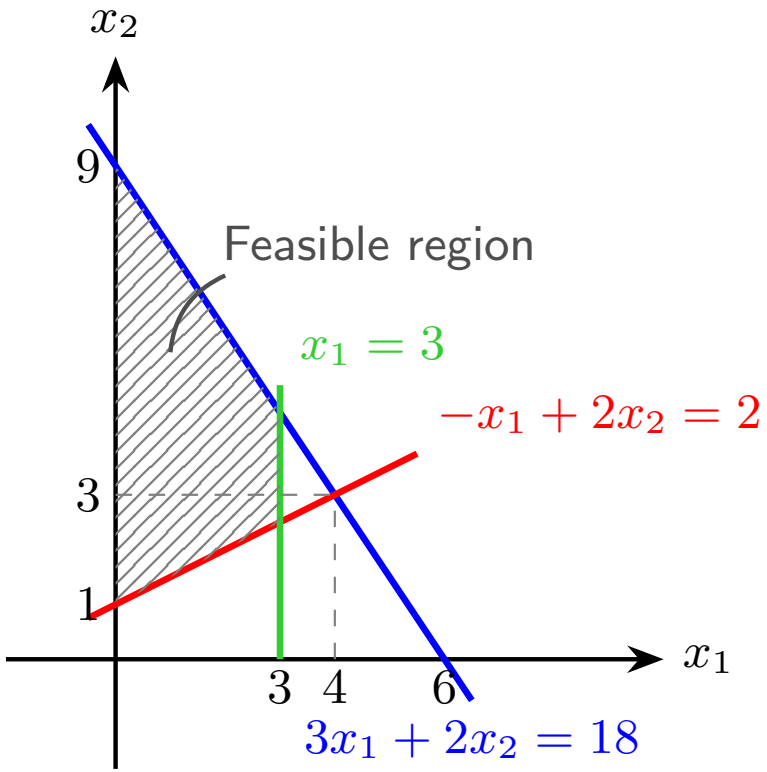
Geometry of Linear Programming

Linear programming:

$$\begin{aligned} \max_{x_1,x_2} \quad & 4x_1 + x_2 \\ \text{s.t.} \quad & x_1 \leq 3 \\ & 3x_1 + 2x_2 \leq 18 \\ & -x_1 + 2x_2 \geq 2 \\ & x_1, x_2 \geq 0 \end{aligned}$$

Optimal solution:

$$x_1^\star = 3 \quad x_2^\star = 4.5$$



Linear constraints define the **feasible region**, which is a convex polytope.

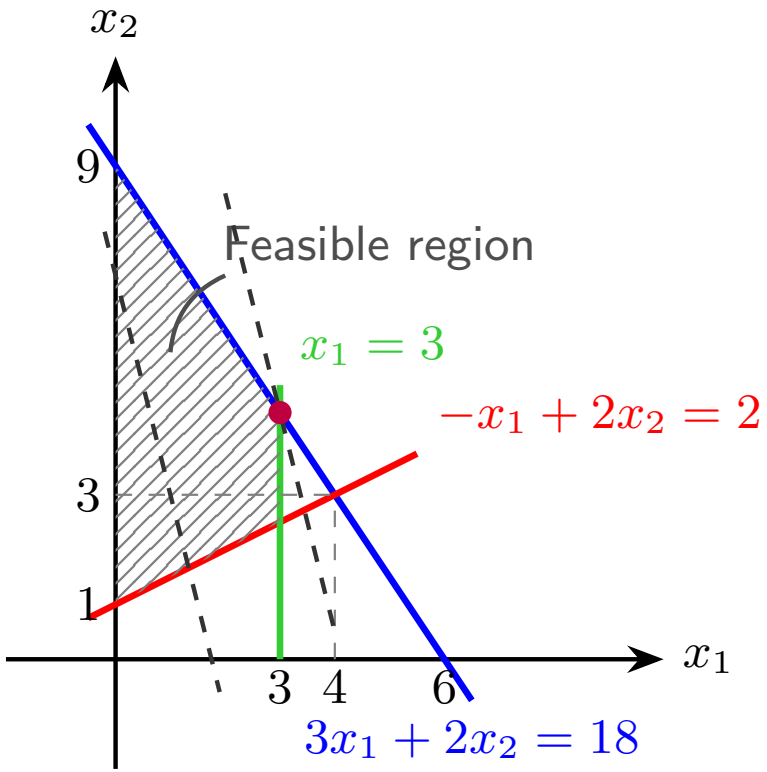
Geometry of Linear Programming

Linear programming:

$$\begin{aligned} \max_{x_1,x_2} \quad & 4x_1 + x_2 \\ \text{s.t.} \quad & x_1 \leq 3 \\ & 3x_1 + 2x_2 \leq 18 \\ & -x_1 + 2x_2 \geq 2 \\ & x_1, x_2 \geq 0 \end{aligned}$$

Optimal solution:

$$x_1^\star = 3 \quad x_2^\star = 4.5$$



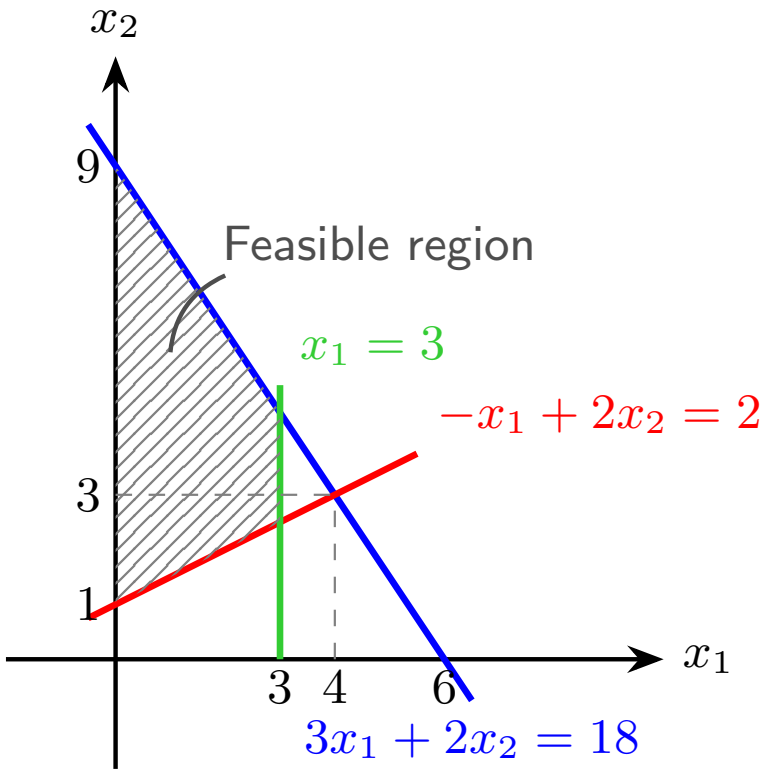
Linear constraints define the **feasible region**, which is a convex polytope.

Geometry of Linear Programming

Linear programming:

$$\begin{aligned} \min_{x_1,x_2} \quad & -x_1 + x_2 \\ \text{s.t.} \quad & x_1 \le 3 \\ & 3x_1 + 2x_2 \le 18 \\ & -x_1 + 2x_2 \ge 2 \\ & x_1, x_2 \ge 0 \end{aligned}$$

Optimal solution?



Linear constraints define the **feasible region**, which is a convex polytope.

Python Programming

```python
import cvxpy as cp

# Define the decision variables
x = cp.Variable(2, nonneg=True)
# Define the objective function
objective = cp.Minimize(-x[0] + x[1])

# Define the constraints
constraints = [
    x[0] <= 3,                          # x1 <= 3
    3*x[0] + 2*x[1] <= 18,              # 3x1 + 2x2 <= 18
    -x[0] + 2*x[1] >= 2                 # -x1 + 2x2 >= 2
]

# Formulate and solve the problem
prob = cp.Problem(objective, constraints)
result = prob.solve()
# Output the results
print(f'Status: {prob.status}')
print(f'Optimal Value (max objective): {prob.value:.4f}')
print(f'Optimal x1: {x.value[0]:.4f}')
print(f'Optimal x2: {x.value[1]:.4f}')
```

ℓ_1-Norm Optimization

Given $\boldsymbol{A} \in \mathbb{R}^{n \times n}$ and $\boldsymbol{b} \in \mathbb{R}^n$, find the best $\boldsymbol{x} \in \mathbb{R}^n$ that minimizes

$$\boldsymbol{\varepsilon} = \boldsymbol{b} - \boldsymbol{A}\boldsymbol{x}$$

- The sum of relative residuals:

$$\underbrace{\|\boldsymbol{\varepsilon}\|_1}_{\ell_1\text{-norm}} = \sum_{i=1}^{n} |\varepsilon_i|$$

- ℓ_1-norm optimization:

$$\begin{aligned} \min_{\boldsymbol{x}} \quad & \sum_{i=1}^{n} |\varepsilon_i| \\ \text{s.t.} \quad & \boldsymbol{\varepsilon} = \boldsymbol{b} - \boldsymbol{A}\boldsymbol{x} \end{aligned}$$

ℓ_1-Norm Optimization

- Linear programming with bound variables $c_i, i = 1, 2, \ldots, n$:

$$\begin{aligned} \min_{\boldsymbol{x},\boldsymbol{c}} \quad & \sum_{i=1}^{n} c_i \\ \text{s.t.} \quad & \boldsymbol{\varepsilon} = \boldsymbol{b} - \boldsymbol{A}\boldsymbol{x} \\ & -c_i \leq \varepsilon_i \leq c_i \qquad i = 1, 2, \ldots, n \\ & c_i \geq 0 \qquad\qquad\quad i = 1, 2, \ldots, n \end{aligned}$$

- Toy example:

$$\boldsymbol{A} = \begin{bmatrix} 3 & 2 \\ -1 & 2 \end{bmatrix} \qquad \boldsymbol{b} = \begin{bmatrix} 18 \\ 2 \end{bmatrix} \quad \Rightarrow \quad \text{Exact solution:} \quad \boldsymbol{x} = \begin{bmatrix} 4 \\ 3 \end{bmatrix}$$

ℓ_1-Norm Optimization

```python
import cvxpy as cp
import numpy as np

n = 2
A = np.array([[3, 2], [-1, 2]])
b = np.array([18, 2])

# Define decision variables
x = cp.Variable(n)
c = cp.Variable(n, nonneg=True)
# Minimize the sum of the bound variables c_i
objective = cp.Minimize(cp.sum(c))
# Define constraints
constraints = [-c <= b - A @ x, b - A @ x <= c]
# Formulate and solve the problem
prob = cp.Problem(objective, constraints)
prob.solve()
# Output results
print(f'Status: {prob.status}')
print(f'Optimal objective value (sum of c): {prob.value}')
print(f'Optimal x:\n{x.value}')
```

ℓ_∞-Norm Optimization

Given $\boldsymbol{A} \in \mathbb{R}^{n\times n}$ and $\boldsymbol{b} \in \mathbb{R}^n$, find the best $\boldsymbol{x} \in \mathbb{R}^n$ that minimizes

$$\boldsymbol{\varepsilon} = \boldsymbol{b} - \boldsymbol{A}\boldsymbol{x}$$

- The worst-case residual:

$$\underbrace{\|\boldsymbol{\varepsilon}\|_\infty}_{\ell_\infty\text{-norm}} = \max\{|\varepsilon_1|, |\varepsilon_2|, \ldots, |\varepsilon_n|\}$$

- ℓ_∞-norm optimization:

$$\begin{aligned} \min_{\boldsymbol{x}} \quad & \max\{|\varepsilon_1|, |\varepsilon_2|, \ldots, |\varepsilon_n|\} \\ \text{s.t.} \quad & \boldsymbol{\varepsilon} = \boldsymbol{b} - \boldsymbol{A}\boldsymbol{x} \end{aligned}$$

ℓ_∞-Norm Optimization

- Linear programming with bound variable c (scalar):

$$\begin{aligned} \min_{\boldsymbol{x},c} \quad & c \\ \text{s.t.} \quad & \boldsymbol{\varepsilon} = \boldsymbol{b} - \boldsymbol{A}\boldsymbol{x} \\ & -c \leq \varepsilon_i \leq c \quad i = 1, 2, \ldots, n \\ & c \geq 0 \end{aligned}$$

- Toy example:

$$\boldsymbol{A} = \begin{bmatrix} 3 & 2 \\ -1 & 2 \end{bmatrix} \qquad \boldsymbol{b} = \begin{bmatrix} 18 \\ 2 \end{bmatrix} \quad \Rightarrow \quad \text{Exact solution:} \quad \boldsymbol{x} = \begin{bmatrix} 4 \\ 3 \end{bmatrix}$$

ℓ_∞-Norm Optimization

```python
import cvxpy as cp
import numpy as np

n = 2
A = np.array([[3, 2], [-1, 2]])
b = np.array([18, 2])

# Define decision variables
x = cp.Variable(n)
c = cp.Variable(1, nonneg=True)
# Minimize the bound variable c
objective = cp.Minimize(c)
# Define constraints
constraints = [-c <= b - A @ x, b - A @ x <= c]
# Formulate and solve the problem
prob = cp.Problem(objective, constraints)
prob.solve()
# Output results
print(f'Status: {prob.status}')
print(f'Optimal objective value: {prob.value}')
print(f'Optimal x:\n{x.value}')
```

Quick Summary

Monday's Class:

- Essential formula of optimization
- What is linear programming?
- Geometric interpretation of linear programming
- Applications to ℓ_1- and ℓ_∞-norm optimization